

From Batch Reduction to Investigative Fixpoint: A DLR-Lambda-2 Experiment on Marker-Free Log Diagnosis

Nikit Phadke

June 2026

Abstract

The first DLR-lambda Loghub experiment showed that typed recursive control can compose distributed log evidence into a ProcessIR causal diagnosis. Its control plan, however, was still batch-shaped: split, map, filter, reduce, output. This paper freezes the next finding. Some incidents are not merely distributed; their later evidence changes the meaning of earlier evidence. In such cases, event recall is not enough. The controller must treat certain facts as frontier items, schedule follow-up retrieval, reconcile contradictions, revise hypotheses, and iterate until the ProcessIR reaches a verified fixpoint. We implement a marker-free Loghub experiment comparing one-pass MapReduce-style DLR-lambda against a typed worklist/fixpoint controller. Both systems recover all observed event nodes. The MapReduce system still fails: it misses the expired-exemption latent condition, mishandles contradiction, fails hypothesis revision, and does not detect the local approval as a false terminal state. The worklist/fixpoint system recovers all event nodes, latent conditions, causal edges, root causes, and terminal-state diagnostics. The result supports a spectrum view: MapReduce-style DLR-lambda is sufficient when evidence is independent and reduction is associative; worklist/fixpoint DLR-lambda is needed when facts generate follow-up obligations and later evidence revises earlier meaning.

1 Introduction

Operational diagnosis is often described as a search over evidence. That is only partly true. In harder incidents, a fact is not merely an observation to store. It is an obligation to investigate. A local service reports success, but the parent workflow may still be invalid. A cached policy is used, but its freshness must be checked. An exemption is claimed, but its validity at transition time must be verified. An authority is asserted, but its domain must be confirmed.

The first DLR-lambda experiment [1] used a simple lambda-style recursive plan over Loghub Linux logs: split the trace into chunks, map chunks to event frames, filter incident evidence, and reduce the frames into ProcessIR. That experiment established a useful lower rung: when evidence is separated but mostly independent, batch-style recursive composition can recover a latent stale-representation root cause.

This paper asks a sharper question:

What happens when a later fact changes the meaning of an earlier fact?

The answer is that MapReduce is no longer the right conceptual center. It is a special case. DLR-lambda-2 is better modeled as typed recursive control over an evidence graph until the relevant motif obligations reach fixpoint. In this paper we implement the intermediate form, a worklist/fixpoint controller. We do not yet make motif obligations a first-class calculus. Instead, we show why the worklist layer is necessary and where the next semantic layer should attach.

2 The Spectrum

DLR-lambda should not be presented as a single execution plan. Different diagnostic problems need different control strength. Table 1 summarizes the emerging spectrum.

Table 1: A spectrum of DLR-lambda control.

Stage	Center	Appropriate when
DLR-lambda-1	MapReduce composition	Evidence is distributed, mostly independent, and reduction is close to associative.
DLR-lambda-2	Worklist/fixpoint investigation	Facts can change the meaning of prior facts and require scheduled follow-up joins.
DLR-lambda-3	Motif obligation calculus	The system must prove that mandatory structural obligations are satisfied, violated, or explicitly unknown.

This is not a replacement story. Many users will only need DLR-lambda-1. A log classification pipeline, document triage system, or retrieval job may only need bounded extraction plus final composition. DLR-lambda-2 is needed when evidence creates unresolved frontier items. DLR-lambda-3 becomes necessary when terminal validity itself must be formalized as an obligation-discharge problem.

3 Background

3.1 DLR ProcessIR

Disentangled Language Representation (DLR) represents traces as typed process diagnoses rather than unstructured summaries. The relevant ProcessIR fields are event frames, latent conditions, typed causal edges, root causes, false terminal states, violated invariants, hypothesis revisions, prescriptions, and confidence. This paper focuses on a specific ProcessIR failure pattern: a local transition reports success while the parent process has not reached a valid terminal state.

3.2 Lambda-Style Control

Lambda-RLM argues that long-context reasoning can be made more reliable by expressing recursive control with typed functional primitives rather than unbounded code generation or a single monolithic prompt [2]. The DLR-lambda experiments borrow this control idea but use ProcessIR as the target representation. Recursive control answers how evidence is traversed. ProcessIR answers what semantic object is recovered.

3.3 MapReduce as a Special Case

The DLR-lambda-1 control plan was:

SPLIT -> MAP -> FILTER -> REDUCE -> OUTPUT

This is useful but structurally limited. It assumes that chunks can be processed mostly independently, that extraction can precede global context, that one reduction pass is enough, and that evidence relevance is known early. Those assumptions break when an extracted fact creates a new question that must be answered before the diagnosis can terminate.

4 DLR-Lambda-2

The DLR-lambda-2 control plan is:

```
INGEST -> EXTRACT -> JOIN -> SCHEDULE FOLLOW-UP
      -> RECONCILE -> FIXPOINT -> VERIFY -> ProcessIR
```

The key difference is that the controller maintains typed evidence state. It can derive hypotheses, schedule new work, weaken or retract prior hypotheses, and stop only when the graph reaches a coverage-checked fixpoint.

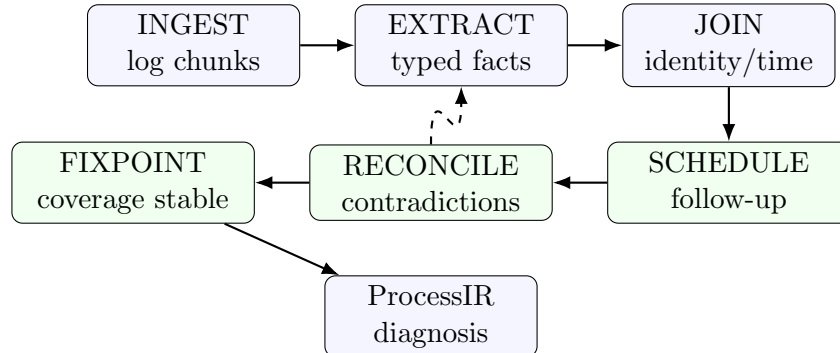


Figure 1: DLR-lambda-2 as typed investigative fixpoint. Reconciliation can schedule more extraction before the ProcessIR is terminal-valid.

5 Experimental Design

5.1 Dataset

The experiment uses the public Loghub Linux sample as a noisy carrier [3, 4, 5]. The original Linux log lines are left intact. Seven marker-free synthetic lines are injected into the trace, producing 2007 total lines. Unlike the first DLR-lambda experiment, the injected evidence does not use an explicit `DLR_LAMBDA` marker. The parser must recover the incident using ordinary textual evidence such as refund IDs, policy versions, exemption identifiers, audit observations, and settlement outcomes.

5.2 Incident

The incident is designed around a meaning-changing later fact. A refund is approved using policy version v17 after policy version v18 exists. However, the approval claims VIP exemption `EX-77`, which could make the apparent policy mismatch valid. The exemption therefore weakens the stale-policy hypothesis and creates a follow-up obligation: was `EX-77` valid at the time of approval? A later line reveals that the exemption expired before approval. The final diagnosis must therefore revise the hypothesis and treat the local approval as a false terminal state.

5.3 Systems Compared

The benchmark compares two systems:

Table 2: Distributed marker-free evidence.

Line	Evidence summary
117	Local policy cache has TTL 1800s and requires current policy before terminal approval.
444	Refund policy changes to version v18, requiring manager review unless an active VIP exemption exists.
809	Worker approves refund R-914 using policy v17 and claims exemption EX-77.
1044	Entitlements service asserts VIP exemption EX-77 for the refund.
1377	Entitlements service states that EX-77 expired before approval.
1600	Audit observes v18 expected, v17 observed, and manager review missing.
1817	Finance blocks settlement because approval was not terminal-valid.

1. **One-pass MapReduce DLR-lambda:** scans chunks once, extracts primary facts, and performs a single reduction into ProcessIR. It does not schedule follow-up retrieval for exemption validity.
2. **Worklist/fixpoint DLR-lambda:** scans chunks, stores typed facts, treats exemption and terminal claims as frontier items, schedules follow-up searches, revises hypotheses, verifies terminal evidence, and emits ProcessIR after fixpoint.

6 Gold Diagnosis

The gold root causes are:

- **lc_stale_policy_representation:** local approval used policy v17 after authoritative policy v18 existed;
- **lc_expired_vip_exemption:** the apparent exemption existed but expired before approval.

The false terminal state is the local approval. It is locally successful, but globally invalid because the policy version is stale, the exemption is expired, the manager-review invariant is missing, and downstream settlement is blocked.

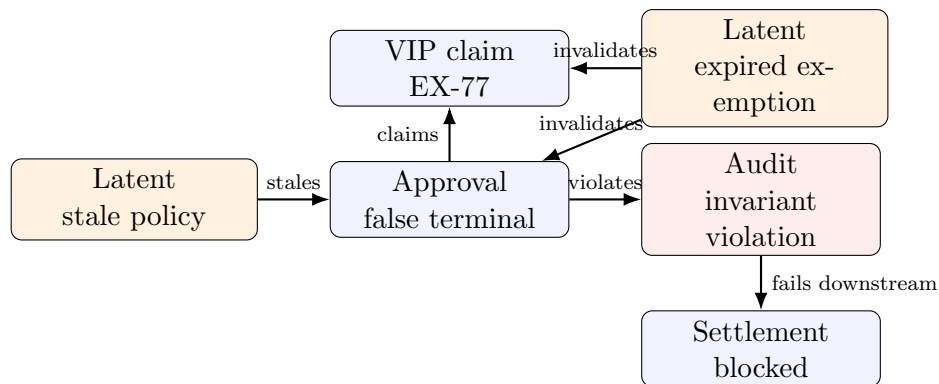


Figure 2: Gold diagnosis. The exemption claim is not an answer; it is a frontier item whose validity must be joined against time and authority evidence.

7 Results

Table 3: System-level scores.

System	Conf.	Event recall	Latent recall	Edge recall	Root cause	Contr.	Rev.	Cov.	False terminal
One-pass MapReduce DLR-lambda	0.62	1.00	0.50	0.83	0.50	no	no	0.67	no
Worklist/fixpoint DLR-lambda	0.88	1.00	1.00	1.00	1.00	yes	yes	1.00	yes

The crucial result is that the one-pass MapReduce system achieves perfect event node recall and still fails. It sees the primary events, including the approval, VIP exemption claim, audit violation, and settlement block. It misses the expired-exemption latent condition because it does not compile the exemption claim into a scheduled follow-up search. As a result, it also misses the hypothesis revision and fails to mark the approval as a false terminal state.

The worklist/fixpoint system passes all reported criteria. It treats the approval’s exemption as unresolved, schedules `follow_exemption_EX-77`, finds the expiration evidence, revises the hypothesis, verifies audit and settlement outcomes, and emits the complete ProcessIR.

Table 4: Worklist trace excerpt.

Step	Action	Scheduled/discovered	Revision
3	Scan approval chunk	Schedules <code>follow_exemption_EX-77</code> and terminal verification	—
4	Scan exemption-claim chunk	Discovers VIP exemption claim	Stale-policy hypothesis weakened
8	Follow exemption	Discovers expired exemption	VIP claim revised to invalid
9	Verify terminal	Audit and settlement already present	Final false-terminal diagnosis stable

8 Interpretation

The experiment shows that seeing all facts is not the same as knowing which facts create obligations. A one-pass reducer can collect the exemption claim and still fail to ask the question it implies. The worklist controller succeeds because it treats the exemption as a frontier item.

The generalized rule is:

Certain facts are not terminal facts. They are obligation-generating facts.

Examples include:

- exemption claimed $\rightarrow$ validate exemption at transition time;
- cached representation used $\rightarrow$ validate freshness;
- authority asserted $\rightarrow$ validate authority domain;
- terminal success reported $\rightarrow$ validate parent invariant;

- identity matched $\rightarrow$ validate aliasing and collision;
- queue accepted job $\rightarrow$ validate capacity and completion.

DLR-lambda-2 does not yet formalize all such obligations. It demonstrates the need for them. Worklist/fixpoint is the operational layer that makes obligation-driven diagnosis possible.

9 Why This Freezes a Distinct Finding

DLR-lambda-1 and DLR-lambda-2 solve different problems. DLR-lambda-1 is not obsolete. It is the right tool when distributed evidence can be independently extracted and safely reduced. DLR-lambda-2 is needed when evidence changes meaning under later joins.

The distinction matters commercially and scientifically. A team building a large-scale summarizer may only need DLR-lambda-1. A team building production incident diagnosis, compliance workflow audit, agentic tool-use validation, or payment reconciliation will often need DLR-lambda-2 because local facts create follow-up obligations and terminal states must be checked against parent invariants.

10 Limitations

This is still a controlled synthetic incident embedded in a real Loghub substrate. The fact extractor is deterministic and schema-specific. The experiment does not claim broad open-ended log understanding. It is a falsification-style probe for the control abstraction. The intended conclusion is narrow: a batch reducer can fail even with perfect observed-event recall when a fact requires follow-up validation and hypothesis revision.

The next step is not merely to add more incidents. It is to make obligations first-class:

```
Fact -> MotifObligation -> ScheduledSearch -> Status
```

That would yield DLR-lambda-3: a motif obligation calculus in which ProcessIR is terminal-valid only when mandatory obligations are satisfied, violated, or explicitly unknown under budget.

11 Conclusion

DLR-lambda should be understood as a spectrum of typed recursive control strength. MapReduce is a useful weak execution plan, not the conceptual center. DLR-lambda-2 shows that diagnosis sometimes requires a typed worklist and fixpoint loop: facts are extracted, joins create or revise hypotheses, follow-up searches are scheduled, contradictions are reconciled, and ProcessIR is emitted only after coverage stabilizes.

The experiment's decisive observation is that one-pass MapReduce recovers all observed event nodes and still fails. The worklist/fixpoint controller succeeds because it recognizes that an exemption is not an answer. It is a new frontier item.

A Appendix: Reproduction

The experiment is implemented in the repository as:

```
npm run run:dlr-lambda-worklist
```

The command downloads the Loghub Linux sample if needed, injects the marker-free synthetic incident, writes the Markdown report, and writes the ProcessIR JSON artifact.

References

- [1] Nikit Phadke. Recursive Log Diagnosis with ProcessIR: A DLR-Lambda Experiment on Loghub. 2026.
- [2] Amartya Roy, Rasul Tutunov, Xiaotong Ji, Matthieu Zimmer, and Haitham Bou-Ammar. The Y-Combinator for LLMs: Solving Long-Context Rot with λ -Calculus. arXiv:2603.20105v1, 2026. <https://arxiv.org/abs/2603.20105>.
- [3] LogPAI. Loghub: A large collection of system log datasets for AI-driven log analytics. <https://github.com/logpai/loghub>.
- [4] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. IEEE International Symposium on Software Reliability Engineering (ISSRE), 2023.
- [5] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R. Lyu. A Large-scale Evaluation for Log Parsing Techniques: How Far are We? ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2024.